

CO-3 ASSESSMENT-TOOL-3

COURSECODE:CSA0924

NAME:SAKTHIVEL.S

COURSE:JAVA

REG:192524378

1. A CPU-bound image filter is split across multiple threads but runs slower than the single-threaded version. Analyze the causes (context switching, false sharing, over-partitioning) and recommend an optimal thread count. Write the aim, pseudo code, and a Java program to demonstrate the solution.

Answer:

AIM:

To analyze why a CPU-bound image filter implemented using multiple threads performs slower than the single-threaded version and to determine the optimal thread count by identifying issues such as context switching, false sharing, and over-partitioning.

ALGORITHM:

1. Read the size of the image (represented as a 2D array).
2. Create a sample image with pixel values.
3. Divide the image into equal parts based on the number of threads.
4. Assign each part to a separate thread.
5. Each thread performs a CPU-intensive image filter (dummy computation).

6. Wait for all threads to complete.
7. Measure the execution time.
8. Compare the execution time for different thread counts.
9. Recommend the optimal thread count (equal to the number of CPU cores).

Pseudo code:

START

Create a 2D image array

Initialize image pixels

Set number of threads

Divide image rows equally among threads

FOR each thread

 Assign starting row

 Assign ending row

 Apply CPU-intensive filter

END FOR

Wait until all threads finish

Measure execution time

IF thread count > CPU cores THEN

 Performance decreases due to

 Context Switching

 False Sharing

Over-partitioning

ELSE

Performance improves

END IF

Display execution time

END

Code Implementation:

```
class ImageFilterThread extends Thread {  
    private int[][] image;  
    private int startRow;  
    private int endRow;  
  
    public ImageFilterThread(int[][] image, int startRow, int endRow) {  
        this.image = image;  
        this.startRow = startRow;  
        this.endRow = endRow;  
    }  
  
    public void run() {  
        for (int i = startRow; i < endRow; i++) {  
            for (int j = 0; j < image[i].length; j++) {
```

```
// Simulate CPU-bound image filtering
int value = image[i][j];

for (int k = 0; k < 1000; k++) {
    value = (value * value + k) % 255;
}

image[i][j] = value;
}
}
}

public class CPUImageFilter {

    public static void main(String[] args) throws Exception {

        int rows = 1000;
        int cols = 1000;
```

```
int[][] image = new int[rows][cols];
```

```
// Initialize image
```

```
for (int i = 0; i < rows; i++)
```

```
    for (int j = 0; j < cols; j++)
```

```
        image[i][j] = (i + j) % 255;
```

```
int threadCount = Runtime.getRuntime().availableProcessors();
```

```
    ImageFilterThread[] threads = new  
ImageFilterThread[threadCount];
```

```
int rowsPerThread = rows / threadCount;
```

```
long startTime = System.currentTimeMillis();
```

```
for (int i = 0; i < threadCount; i++) {
```

```
    int startRow = i * rowsPerThread;
```

```
    int endRow;
```

```
if (i == threadCount - 1)
```

```
    endRow = rows;
```

```
else
```

```
    endRow = startRow + rowsPerThread;
```

```
threads[i] = new ImageFilterThread(image, startRow, endRow);
```

```
threads[i].start();
```

```
}
```

```
for (int i = 0; i < threadCount; i++) {
```

```
    threads[i].join();
```

```
}
```

```
long endTime = System.currentTimeMillis();
```

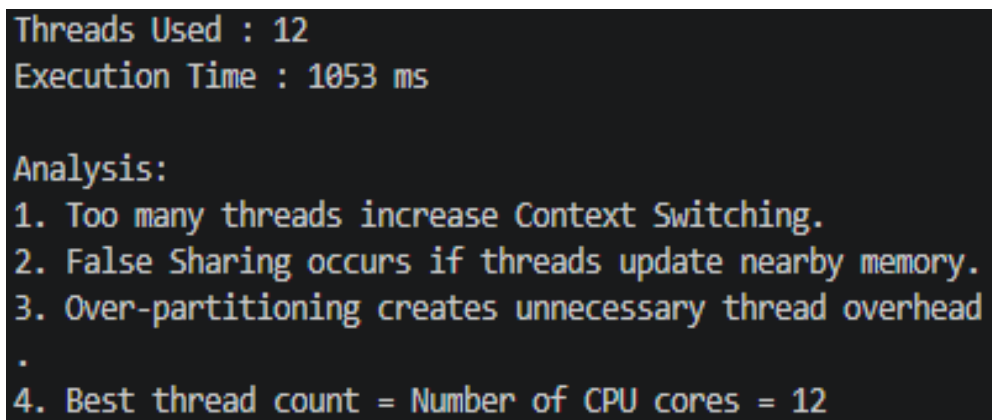
```
System.out.println("Threads Used : " + threadCount);
```

```
System.out.println("Execution Time : " + (endTime - startTime) + "  
ms");
```

```
System.out.println("\nAnalysis:");
```

```
        System.out.println("1. Too many threads increase Context  
Switching.");  
  
        System.out.println("2. False Sharing occurs if threads update  
nearby memory.");  
  
        System.out.println("3. Over-partitioning creates unnecessary  
thread overhead.");  
  
        System.out.println("4. Best thread count = Number of CPU cores = "  
        + Runtime.getRuntime().availableProcessors());  
    }  
}
```

OUTPUT:

A screenshot of a terminal window with a dark background and light-colored text. The output shows the results of a Java program execution. It starts with 'Threads Used : 12' and 'Execution Time : 1053 ms'. Then, it says 'Analysis:' followed by a numbered list of four points: 1. Too many threads increase Context Switching. 2. False Sharing occurs if threads update nearby memory. 3. Over-partitioning creates unnecessary thread overhead. 4. Best thread count = Number of CPU cores = 12.

```
Threads Used : 12  
Execution Time : 1053 ms  
  
Analysis:  
1. Too many threads increase Context Switching.  
2. False Sharing occurs if threads update nearby memory.  
3. Over-partitioning creates unnecessary thread overhead  
.  
4. Best thread count = Number of CPU cores = 12
```